

Санкт-Петербургский государственный университет

Кафедра информационно-аналитических систем

Группа 23.Б10-мм

Реализация алгоритма валидации
последовательных зависимостей в проекте
Desbordante

Закарлюка Иван Владимирович

Отчёт по учебной практике
в форме «Решение»

Научный руководитель:
ассистент кафедры ИАС, Чернышев Г. А.

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	5
2. Обзор	6
2.1. Основные определения	6
2.2. Примеры	7
2.3. Анализ существующих решений	9
3. Описание решения	10
3.1. Описание алгоритма	10
3.1.1. Сведение к задаче о подпоследовательности . . .	10
3.1.2. Пояснительный пример	11
3.1.3. Динамическое программирование	11
3.1.4. Случай точного интервала ($G_1 = G_2 = G$)	14
3.1.5. Случай интервала ($G_1 < G_2$)	14
3.1.6. Восстановление пути	15
3.2. Интеграция алгоритма в Desbordante	16
3.2.1. Вспомогательные структуры данных	18
3.3. Python-привязки	19
3.4. Пример использования	20
4. Эксперимент	23
4.1. Результаты	24
4.2. Анализ результатов	24
Заключение	26
Список литературы	27

Введение

В современном мире объёмы данных, генерируемых организациями и техническими системами, непрерывно возрастают. Журналы событий, показания датчиков, финансовые транзакции — всё это формирует массивы упорядоченных по времени записей, анализ которых позволяет выявлять закономерности, обнаруживать аномалии и принимать обоснованные решения. Например, в сетевом мониторинге периодический опрос оборудования порождает временные ряды, в которых ожидается определённая регулярность интервалов между измерениями. Отклонения от этой регулярности могут свидетельствовать о потерях данных, сбоях оборудования или ошибках конфигурации.

Одним из подходов к систематическому анализу подобных закономерностей является профилирование данных — процесс извлечения метаданных, то есть сведений о самих данных, из их массивов. Научоёмкое профилирование данных направлено на выявление скрытых взаимосвязей и структурных свойств, описываемых с помощью математически определённых примитивов. Примером такого примитива является последовательная зависимость (Sequential Dependency, SD).

SD — это примитив, описывающий ограничения на разность значений атрибута между последовательными записями в упорядоченном наборе данных [4]. SD задаётся в виде $X \rightarrow [G_1, G_2] Y$, что означает: при сортировке данных по атрибуту X разность значений атрибута Y между любыми двумя соседними записями должна лежать в интервале $[G_1, G_2]$. Этот примитив особенно полезен для анализа временных рядов и упорядоченных данных, где важна не абсолютная величина значений, а характер их изменения.

Desbordante [1] — это платформа для высокопроизводительного научноёмкого профилирования данных с открытым исходным кодом. В настоящее время в Desbordante реализованы алгоритмы поиска и валидации для множества примитивов, однако алгоритм валидации последовательных зависимостей отсутствует.

Целью настоящей работы является проектирование и реализация

алгоритма валидации SD в ядре Desbordante на языке C++, создание Python-привязок для удобного использования алгоритма, а также разработка наглядного примера использования.

1. Постановка задачи

Целью работы является реализация алгоритма валидации SD в проекте Desbordante, а также создание наглядных примеров его использования. Для достижения поставленной цели были сформулированы следующие задачи:

1. Изучить формальное определение SD
2. Реализовать алгоритм валидации SD на языке программирования C++ в ядре проекта Desbordante
3. Реализовать Python-привязки для алгоритма валидации
4. Создать пример использования валидатора SD на языке Python

2. Обзор

2.1. Основные определения

Все нижеприведённые определения основаны на статье [4].

Определение 1. Пусть R — отношение (таблица) с множеством атрибутов A , а I — экземпляр отношения R , то есть конечный набор кортежей. Каждый атрибут $B \in A$ имеет область определения $dom(B)$ — множество допустимых значений. Значение атрибута B в кортеже t обозначается $t[B]$.

Определение 2. Пусть $X, Y \in A$ — два атрибута отношения R , причём $dom(Y) \subseteq R$ (значения Y числовые). SD записывается в виде:

$$X \rightarrow [G_1, G_2] Y,$$

где $G_1, G_2 \in R$, $0 \leq G_1 \leq G_2$ (или $G_2 = +\infty$). Атрибут X задаёт порядок сортировки кортежей, а параметры G_1 и G_2 определяют допустимый диапазон разности значений атрибута Y между любыми двумя последовательными (в порядке X) кортежами.

Определение 3. SD $\phi: X \rightarrow [G_1, G_2] Y$ удерживается (holds) на экземпляре I , если при упорядочении кортежей по значениям атрибута X для каждой пары последовательных кортежей t_i, t_{i+1} выполнено:

$$G_1 \leq t_{i+1}[Y] - t_i[Y] \leq G_2.$$

Иными словами, разность значений Y между каждыми двумя соседними записями лежит в заданном интервале.

Определение 4. Если SD ϕ не удерживается на экземпляре I в точности, степень её нарушения измеряется через минимальное число операций $OPS(\phi, I)$, необходимых для приведения данных к виду, на котором ϕ удерживается. Допустимы два вида операций:

- Удаление — исключение кортежа из последовательности (например, если его значение Y слишком близко к соседнему или слишком далеко от обоих соседей)

- Вставка — добавление виртуального кортежа между двумя существующими, чтобы покрыть чрезмерно большой разрыв в значениях Y несколькими допустимыми шагами

Определение 5. Уверенность (Confidence) SD ϕ на экземпляре I определяется как:

$$\text{Confidence}(\phi, I) = \frac{N - \text{OPS}(\phi, I)}{N},$$

где $N = |I|$ — число кортежей. Уверенность принимает значения от 0 (хотя точного нуля не может быть, ведь можно удалить все элементы кроме одного) до 1 (зависимость удерживается без изменений, OPS = 0).

Определение 6. В рамках настоящей работы введём понятие нарушения (Violation) — конкретной операции, входящей в оптимальный набор из OPS операций. Нарушение может быть одного из двух типов:

- Удаление (Deletion) — указание на кортеж, который необходимо исключить из последовательности. Это происходит, когда значение Y кортежа не может быть связано с соседними кортежами через допустимый интервал $[G_1, G_2]$
- Вставка (Insertion) — указание на разрыв между двумя последовательными кортежами оптимальной подпоследовательности, который слишком велик для одного шага в интервале $[G_1, G_2]$ и требует добавления промежуточных записей

2.2. Примеры

Пример 1. Рассмотрим таблицу 1, содержащую записи сетевого мониторинга: номер опроса (PollNum) и время в секундах (Time). Предположим, что сетевое оборудование должно опрашиваться каждые 9–11 секунд. Это ожидание выражается как SD:

$$\phi: \text{PollNum} \rightarrow [9, 11] \text{ Time}.$$

Таблица 1: Журнал сетевого мониторинга

PollNum	Time
1	10
2	20
3	30
4	70
5	80
6	90
7	92
8	94
9	97
10	100

При сортировке по PollNum последовательные разности значений Time составляют: 10, 10, 40, 10, 10, 2, 2, 3, 3. Разность 40 (между строками 3 и 4) превышает верхнюю границу $G_2 = 11$ — для её покрытия необходимы вставки. Разности 2, 2, 3 (между строками 6–9) меньше нижней границы $G_1 = 9$ — соответствующие строки подлежат удалению.

Оптимальное решение: удалить строки 7, 8, 9 (3 удаления) и вставить 3 виртуальные записи между строками 3 и 4, чтобы покрыть разрыв 40 шагами по $[9, 11]$. Итого OPS = 6, уверенность:

$$\text{Confidence}(\phi, I) = \frac{10 - 6}{10} = 0.4$$

Пример 2. Рассмотрим SD с точным интервалом: $\text{PollNum} \rightarrow [10, 10]$ Time. Здесь последовательные разности Time должны составлять ровно 10. На данных таблицы 1 первые три записи (1, 10), (2, 20), (3, 30) удовлетворяют зависимости ($20 - 10 = 10$, $30 - 20 = 10$), но запись (4, 70) нарушает её, поскольку $70 - 30 = 40 \neq 10$.

Пример 3. Рассмотрим SD с бесконечным верхним пределом: $\text{PollNum} \rightarrow [5, +\infty)$ Time. Здесь требуется лишь, чтобы значения Time возрастали не менее чем на 5 между последовательными записями. Малые разности (2, 2, 3) по-прежнему будут нарушениями (меньше $G_1 =$

5), а большой скачок 40 допустим, поскольку $40 \geq 5$.

2.3. Анализ существующих решений

В статье [4] предложена формализация понятия последовательной зависимости и описаны алгоритмы поиска SD в данных. Однако задача валидации конкретной заданной пользователем SD — с вычислением уверенности, минимального числа операций и выявлением конкретных нарушений — в указанной работе подробно не рассматривается.

Проект Desbordante реализует множество примитивов профилирования данных. На момент начала работы алгоритм валидации SD в Desbordante отсутствовал. По результатам проведённого автором обзора, готовых инструментов для валидации SD в сторонних проектах также не обнаружено.

3. Описание решения

3.1. Описание алгоритма

Задача валидации $SD\ X \rightarrow [G_1, G_2]\ Y$ на экземпляре I сводится к следующей задаче: найти минимальное количество операций (вставок и удалений), необходимых для приведения последовательности значений Y (при сортировке по X) к виду, удовлетворяющему ограничению на интервал $[G_1, G_2]$.

3.1.1. Сведение к задаче о подпоследовательности

Пусть данные отсортированы по атрибуту X . Обозначим соответствующие значения атрибута Y как $a_1, a_2, \dots, a_N$. Задача состоит в том, чтобы выбрать из этой последовательности такую подпоследовательность, что суммарное число операций — удалений невошедших элементов и вставок для покрытия разрывов — минимально. Стоит заметить, что это не то же самое, что поиск подпоследовательности максимальной длины: более длинная подпоследовательность требует меньше удалений, но может содержать большие разрывы, требующие многочисленных вставок.

Элементы, не вошедшие в выбранную подпоследовательность, представляют собой удаления: это записи, которые необходимо исключить, чтобы зависимость выполнялась. Если переход между двумя соседними элементами подпоследовательности требует более одного шага, промежуточные шаги интерпретируются как вставки — записи, которые нужно добавить для покрытия разрыва.

Стоимость покрытия расстояния $d = a_{i_{j+1}} - a_{i_j}$ определяется как функция $dcost(d)$ — минимальное целое $k \geq 1$, для которого выполнено $k \cdot G_1 \leq d \leq k \cdot G_2$. Если такого k не существует (например, при $d < 0$), переход невозможен и элементы не могут быть соседями в подпоследовательности. Значение $k = 1$ означает, что переход покрывается одним шагом без вставок; каждый шаг сверх первого ($k - 1$) соответствует одной вставке.

3.1.2. Пояснительный пример

Рассмотрим последовательность $a = (10, 20, 30, 70, 80)$ и SD с параметрами $G_1 = 9$, $G_2 = 11$.

Разности между соседними элементами: 10, 10, 40, 10. Первые две разности (10) и последняя (10) лежат в интервале $[9, 11]$, то есть $\text{dcost}(10) = 1$ — переход допустим без вставок. Однако разность 40 между элементами 30 и 70 слишком велика для одного шага: $\text{dcost}(40) = 4$, так как $4 \cdot 9 = 36 \leq 40 \leq 4 \cdot 11 = 44$. Это означает, что для покрытия разрыва потребовалось бы $4 - 1 = 3$ вставки.

Возможны несколько вариантов восстановления зависимости:

- Оставить все пять элементов и вставить три виртуальные записи между 30 и 70 — итого 3 операции
- Удалить элемент 70 (1 удаление), после чего разность $80 - 30 = 50$, и $\text{dcost}(50) = 5$ ($5 - 1 = 4$ вставки) — итого 5 операций
- Удалить три первых элемента — 3 операции
- Удалить два последних элемента — 2 операции

Последний вариант лучше. Алгоритм находит оптимальное решение с помощью динамического программирования.

3.1.3. Динамическое программирование

Определим две функции на множестве позиций $\{1, \dots, N\}$:

- $T(i)$ — минимальная суммарная стоимость (число операций удаления и вставки) для обработки первых i элементов последовательности, при условии, что i -й элемент входит в оптимальную подпоследовательность. Иными словами, $T(i)$ — это стоимость лучшего решения, в котором a_i — последний сохранённый элемент;
- $\text{OPS}(i)$ — минимальная суммарная стоимость для обработки первых i элементов без каких-либо ограничений на то, входит ли a_i в подпоследовательность.

Значение $T(i)$ вычисляется как минимум из двух вариантов. Первый — удалить все предшествующие элементы, начав подпоследовательность с a_i ; стоимость такого решения равна $i - 1$ (удаление элементов $a_1, \dots, a_{i-1}$). Второй — продолжить подпоследовательность от некоторого ранее сохранённого элемента a_j ($j < i$): при этом все элементы между a_j и a_i удаляются ($(i - 1 - j)$ удалений), а разрыв между a_j и a_i покрывается вставками ($\text{dcost}(a_i - a_j) - 1$ вставок). Рекуррентное соотношение:

$$T(i) = \min \left(i - 1, \min_{\substack{j < i, \\ \text{dcost}(a_i - a_j) \neq -1}} \{T(j) + (i - 1 - j) + (\text{dcost}(a_i - a_j) - 1)\} \right). \quad (1)$$

Здесь минимум по j берётся только по тем позициям, для которых переход от a_j к a_i допустим (то есть $\text{dcost}(a_i - a_j) \neq -1$). Слагаемое $(i - 1 - j)$ отвечает за удаление промежуточных элементов, а $(\text{dcost}(a_i - a_j) - 1)$ — за вставки.

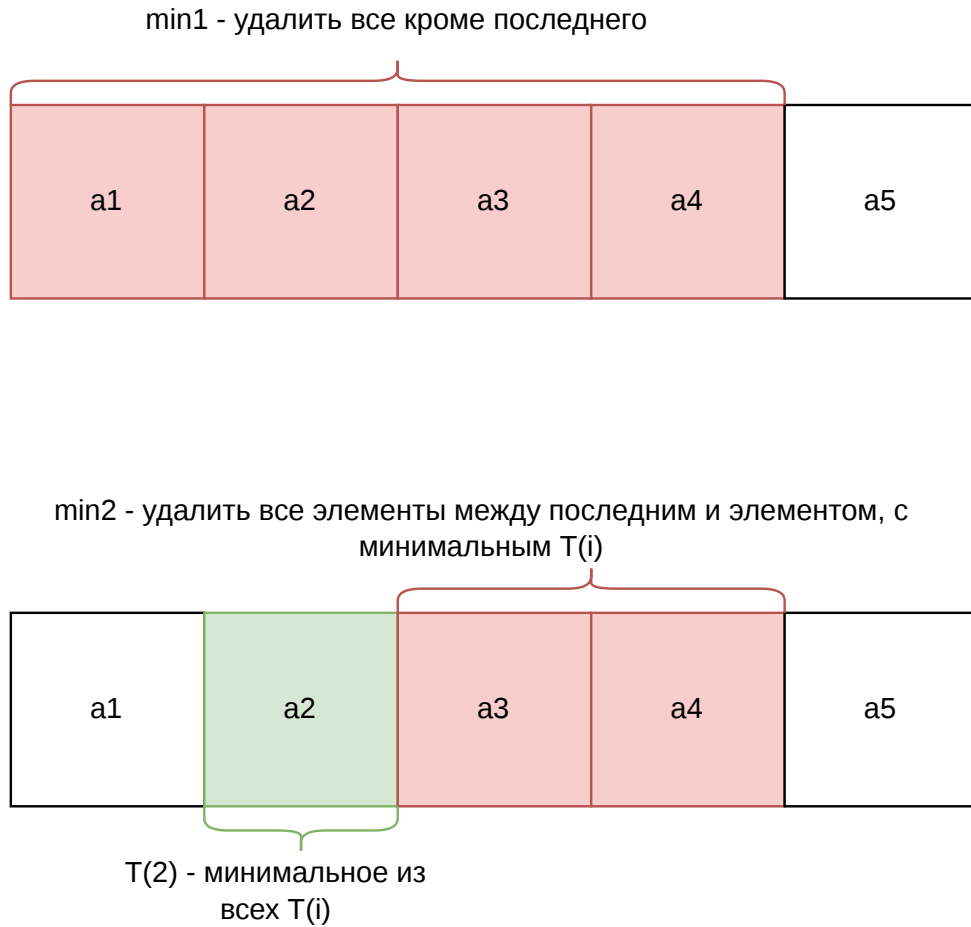


Рис. 1: Визуализация min1 и min2

Значение $OPS(i)$ выбирается как лучший из двух вариантов: либо a_i входит в подпоследовательность (стоимость $T(i)$), либо a_i удаляется (стоимость $OPS(i - 1) + 1$):

$$OPS(i) = \min(T(i), OPS(i - 1) + 1). \quad (2)$$

Ответом задачи является $OPS(N)$ — минимальное число операций для всей последовательности.

Прямое вычисление формулы (1) требует для каждого i перебора всех предшествующих j , что даёт общую сложность $O(N^2)$. Для больших наборов данных это неприемлемо, поэтому применяются специализированные структуры данных, позволяющие находить оптимального

предшественника за $O(\log N)$.

3.1.4. Случай точного интервала ($G_1 = G_2 = G$)

Когда допустимый интервал вырождается в одну точку, разность между любыми двумя последовательными элементами подпоследовательности должна быть строго кратна G . Из этого следует ключевое наблюдение: два элемента a_j и a_i могут быть соседями в подпоследовательности только тогда, когда $a_j \bmod G = a_i \bmod G$, то есть они принадлежат одному классу эквивалентности по остатку от деления на G .

Алгоритм группирует все элементы по классам эквивалентности. Внутри каждого класса задача сводится к поиску оптимального предшественника среди элементов с меньшим значением. Для этого используется дерево Фенвика (Binary Indexed Tree) — структура данных, поддерживающая запросы минимума на префиксе и точечные обновления за $O(\log N)$.

Ключевая идея ускорения состоит в преобразовании ключа. Подставив $\text{dcost}(a_i - a_j) = (a_i - a_j)/G$ в формулу (1), можно выделить часть, зависящую только от j :

$$T(j) - j - \lfloor a_j/G \rfloor.$$

Именно это значение хранится в дереве Фенвика. При обработке элемента a_i достаточно выполнить один запрос минимума на префиксе, чтобы найти лучшего предшественника за $O(\log N)$.

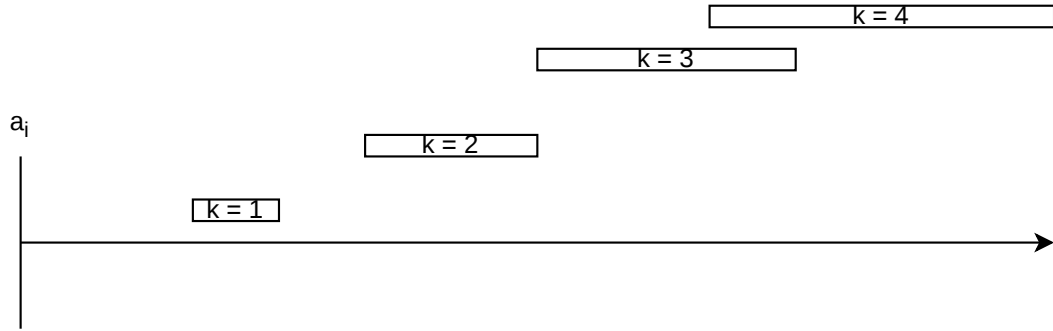
3.1.5. Случай интервала ($G_1 < G_2$)

В общем случае, когда $G_1 < G_2$, допустимые предшественники элемента a_i определяются набором полос (bands) значений. Для каждого числа шагов $k = 1, 2, 3, \dots$ полоса допустимых значений a_j задаётся неравенствами:

$$a_i - k \cdot G_2 \leq a_j \leq a_i - k \cdot G_1.$$

При $k = 1$ это элементы, от которых можно перейти к a_i за один шаг;

при $k = 2$ — за два шага (с одной вставкой) и так далее. С ростом k ширина каждой полосы ($k \cdot (G_2 - G_1)$) увеличивается, и при $k \cdot (G_2 - G_1) \geq G_1$ полосы начинают перекрываться — с этого момента все оставшиеся значения a_j покрываются одним объединённым интервалом.



Для эффективного поиска минимума по полосам применяется дерево отрезков над сжатыми координатами уникальных значений. Ключом в дереве является $T(j) - j - a_j/G_2$ (или $T(j) - j$ при $G_2 = +\infty$). Алгоритм перебирает полосы по возрастающим k и для каждой выполняет запрос минимума на соответствующем отрезке дерева. Как только полосы сливаются, оставшаяся часть покрывается одним запросом. Благодаря этому суммарное число запросов к дереву на всех элементах составляет $O(N)$, а общая сложность — $O(N \log N)$.

Особого рассмотрения заслуживает случай $G_1 = 0$: здесь допускаются нулевые разности, то есть элементы с одинаковым значением Y могут следовать друг за другом без вставок. В этом случае, помимо поиска по полосам, для каждого элемента a_i дополнительно проверяются все ранее обработанные элементы с тем же значением Y . Стоимость перехода от такого элемента a_j составляет $T(j) + (i - 1 - j)$ (только удаления, без вставок). Этот третий вариант учитывается при вычислении $T(i)$ наравне с вариантами «удалить всё» и «перейти через полосы».

3.1.6. Восстановление пути

Помимо минимального числа операций, алгоритм восстанавливает конкретный набор нарушений. Для этого в процессе вычисления $T(i)$

для каждого элемента сохраняется индекс оптимального предшественника $\text{prev}(i)$, а также флаг, указывающий, было ли значение $\text{OPS}(i)$ получено из $T(i)$ или из $\text{OPS}(i - 1) + 1$.

После вычисления $\text{OPS}(N)$ выполняется обратный проход от последнего элемента к первому. На каждом шаге:

- если текущий элемент не входит в оптимальную подпоследовательность (значение OPS получено через удаление), он добавляется в список нарушений как удаление
- если текущий элемент входит в подпоследовательность, алгоритм переходит к его предшественнику $\text{prev}(i)$; все промежуточные элементы помечаются как удаления, а если $\text{dcost}(a_i - a_{\text{prev}(i)}) > 1$, формируется запись о вставке с указанием минимального и максимального числа промежуточных записей

3.2. Интеграция алгоритма в Desbordante

Рассмотрим результат настоящей работы — реализацию алгоритма валидации SD в ядре Desbordante.

Реализация разделена на несколько классов, структура которых представлена на диаграмме (рис. 2).

Класс `SDVerifier`, унаследованный от абстрактного класса `Algorithm`, является главным классом валидатора. Он отвечает за:

- Загрузку данных (`LoadDataInternal()`) — считывание входной таблицы в массив строк `raw_data_`
- Регистрацию параметров (`RegisterOptions()`) — определение опций `lhs_indices`, `rhs_indices`, `g1`, `g2` и необязательного параметра `indices` для подмножества строк
- Выполнение валидации (`ExecuteInternal()`) — основной метод, выполняющий последовательные этапы:
 1. Разбор числовых значений из строковых данных
 2. Сортировку кортежей по значениям LHS (с разрешением совпадений по RHS)
 3. Запуск алгоритма вычисления минимального числа операций (`CalculateOps()`)
 4. Вычисление метрики уверенности

Нарушения представлены типом `SDViolation`, который является вариантным типом (`std::variant`) из двух структур:

- `SDDeletion` — содержит поле `row_idx` (индекс строки, подлежащей удалению)
- `SDInsertion` — содержит индексы строк слева и справа от разрыва (`left_row_idx`, `right_row_idx`), их значения (`val_left`, `val_right`), а также минимальное и максимальное число вставок (`min_insertions`, `max_insertions`).

3.2.1. Вспомогательные структуры данных

Для эффективного выполнения алгоритма реализованы два класса вспомогательных структур данных в пространстве имён `algos::sd_verifier::ds`:

- SegmentTree — дерево отрезков над сжатыми координатами, хранящее узлы типа RmqNode с полями: ключ сравнения val (вычисляемый как $T(j) - j - a_j/G_2$), ранг в сжатых координатах, исходный индекс и значение $T(j)$. Поддерживает операции Update и Query за $O(\log N)$;
- Fenwick — дерево Фенвика, хранящее узлы типа CostNode с ключом $T(j) - j - \lfloor a_j/G \rfloor$. Используется в случае точного интервала для поиска оптимального предшественника в пределах одного класса эквивалентности за $O(\log N)$.

3.3. Python-привязки

Для интеграции алгоритма с Python реализованы привязки с помощью библиотеки `rybind11` [2]. Функция `BindSDVerification()` создаёт подмодуль `sd_verification` в основном Python-модуле и привязывает ключевые классы и методы:

- Класс `SDDeletion` с полем `row_idx` — индекс строки, подлежащей удалению;
- Класс `SDInsertion` с полями: `left_row_idx`, `right_row_idx`, `val_left`, `val_right`, `min_insertions`, `max_insertions`;
- Класс `SDVerifier` (привязанный через `BindPrimitiveNoBase`) со следующими методами:
 - `get_ops()` — возвращает минимальное число операций;
 - `get_violations()` — возвращает список нарушений;
 - `get_error()` — возвращает метрику ошибки ($1 - \text{Confidence}$);
 - `get_confidence()` — возвращает уверенность;
 - `holds(error)` — проверяет, удерживается ли SD с заданным порогом ошибки.

Привязка реализуется через вызов `BindPrimitiveNoBase<SDVerifier>()`, что обеспечивает автоматическую регистрацию алгоритма в системе опций Desbordante и доступ к стандартным методам `load_data()` и `execute()`.

Параметры алгоритма задаются при вызове `execute()`:

- `lhs_indices` — список индексов столбцов LHS;
- `rhs_indices` — список индексов столбцов RHS;
- `g1` — нижняя граница интервала;
- `g2` — верхняя граница интервала (-1 для бесконечности);
- `indices` (необязательный) — подмножество индексов строк для валидации.

3.4. Пример использования

Ниже приведён пример использования библиотеки Desbordante для валидации SD на данных сетевого мониторинга. Код загружает таблицу `network_logs.csv` (таблица 2), выполняет проверку зависимости и визуализирует результаты.

Таблица 2: Данные сетевого мониторинга (`network_logs.csv`)

PollNum	1	2	3	4	5	6	7	8	9	10	11	12	13
Time	10	20	30	70	80	90	92	94	97	100	110	119	130

PollNum	14	15	16	17	18	19	20	21	22	23	24	25	26
Time	150	160	165	167	170	180	190	240	250	260	270	280	390

В данном примере проверяется, опрашивается ли сетевое оборудование с ожидаемой периодичностью (каждые 9–11 секунд). Зависимость формулируется как SD: $\text{PollNum} \rightarrow [9, 11] \text{ Time}$.

Код для валидации с использованием библиотеки Desbordante выглядит следующим образом:

```
import pandas as pd
import desbordante

TABLE_PATH = 'network_logs.csv'

table = pd.read_csv(TABLE_PATH)
algo = desbordante.sd_verification.algorithms.SDVerifier()
algo.load_data(table=table)
algo.execute(g1=9.0, g2=11.0, lhs_indices=[0], rhs_indices=[1])

confidence = algo.get_confidence()
ops = algo.get_ops()
holds = algo.holds(0.0)

print(f"SD: PollNum -> [9.0, 11.0] Time")
print(f"SD strictly holds: {holds}")
print(f"Operations needed (OPS): {ops}")
print(f"Confidence: {confidence:.4f}")
violations = algo.get_violations()
for i, v in enumerate(violations, start=1):
    if isinstance(v, desbordante.sd_verification.SDDeletion):
        print(f"DELETION: Row index {v.row_idx} must be deleted.")
    elif isinstance(v, desbordante.sd_verification.SDInsertion):
        print("INSERTION:")
        print(f"Rows: {v.left_row_idx} {v.right_row_idx}")
        print(f"Values: {v.val_left} -> {v.val_right}.")
        print(f"min: {v.min_insertions}, max: {v.max_insertions}")
```

Результат выполнения:

SD: PollNum $\rightarrow$ [9.0 , 11.0] Time

SD strictly holds: False

Operations needed (OPS): 14

Confidence: 0.4615

INSERTION:

Rows: 2 3

Values: 30.0 $\rightarrow$ 70.0.

min: 3, max: 3

DELETION: Row index 6 must be deleted.

DELETION: Row index 7 must be deleted.

DELETION: Row index 8 must be deleted.

INSERTION:

Rows: 12 13

Values: 130.0 $\rightarrow$ 150.0.

min: 1, max: 1

DELETION: Row index 15 must be deleted.

DELETION: Row index 16 must be deleted.

INSERTION:

Rows: 19 20

Values: 190.0 $\rightarrow$ 240.0.

min: 4, max: 4

DELETION: Row index 25 must be deleted.

Из результатов видно, что алгоритм обнаружил 14 операций: 6 удалений строк (строки 6, 7, 8, 15, 16, 25), содержащих аномально частые измерения или выбросы, и 8 вставок в местах, где данные были утрачены (разрывы между строками 2–3, 12–13, 19–20). Уверенность составила 0.4615, что указывает на существенные нарушения регулярности опроса.

4. Эксперимент

Для экспериментальной оценки производительности алгоритма валидации SD использовался набор данных IoT Telemetry Data [3], содержащий 405 184 записи с измерениями датчиков (температура, влажность, концентрация газов и др.). Каждая запись содержит временную метку (поле *ts*) в формате Unix time. Средний интервал между последовательными измерениями составляет 1.71с, медиана — 1.55с.

Эксперименты проводились на подмножествах данных различного размера (100000, 200000, полный набор 405184 строк) с различными значениями параметров G_1 и G_2 . Для каждой комбинации выполнялось по 20 итераций (с одной прогревочной итерацией), измерялось время выполнения, вычислялись метрики уверенности и минимального числа операций (OPS).

Тестирование проводилось на следующей вычислительной системе:

- Операционная система: NixOS, ядро Linux 6.12.48
- Процессор: AMD Ryzen 7 6800H
- Оперативная память: 16 ГБ

4.1. Результаты

Таблица 3: Результаты эксперимента: время выполнения и метрики

G_1	G_2	Строк	Среднее (мс)	Std dev	Min	Max	OPS
1	2	100 000	63.57	0.88	62.57	66.24	71 253
1	2	200 000	129.69	1.25	127.62	131.80	142 393
1	2	405 184	266.82	1.47	262.76	269.34	288 652
1	3	100 000	54.78	0.49	54.39	56.43	49 942
1	3	200 000	112.30	0.41	111.74	113.31	99 638
1	3	405 184	260.89	1.16	258.81	263.37	201 813
0	5	100 000	49.85	0.36	49.57	51.24	0
0	5	200 000	102.41	0.38	101.77	103.36	0
0	5	405 184	210.99	0.75	209.90	212.33	1
0.5	2.5	100 000	55.11	0.29	54.87	56.30	44 963
0.5	2.5	200 000	112.94	0.44	112.41	113.99	89 556
0.5	2.5	405 184	262.33	0.90	260.64	263.61	181 055
1	1	100 000	40.37	0.42	40.01	41.94	99 999
1	1	200 000	85.09	0.87	84.45	88.05	199 999
1	1	405 184	227.18	2.48	223.80	232.35	405 183
0	$+\infty$	100 000	49.03	0.44	48.80	50.81	0
0	$+\infty$	200 000	100.86	0.59	100.34	103.05	0
0	$+\infty$	405 184	208.66	0.77	207.42	210.25	0

4.2. Анализ результатов

По результатам эксперимента можно сделать вывод, что производительность соответствует теоретической оценке. Время выполнения растёт почти линейно с увеличением размера данных: при удвоении количества строк время увеличивается в 2–2.5 раза, что соответствует оценке сложности $O(N \log N)$.

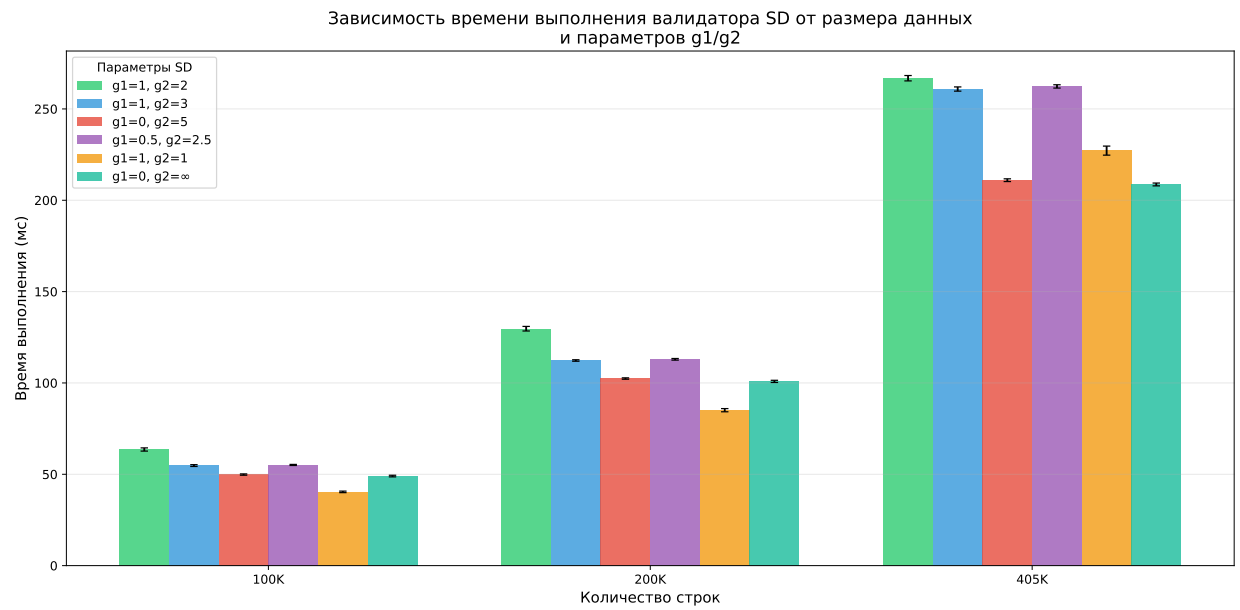


Рис. 3: Зависимость времени выполнения валидатора SD от размера данных и параметров G_1, G_2

На рисунке 3 представлена визуализация результатов в виде столбчатой диаграммы, демонстрирующей влияние различных параметров на время выполнения.

Эксперименты подтвердили работоспособность реализованного алгоритма и его пригодность для практического использования на реальных данных.

Заключение

В рамках данной работы были выполнены следующие задачи:

1. Проведен обзор предметной области, изучено формальное определение последовательных зависимостей и проанализированы существующие решения в проекте Desbordante
2. Спроектирован алгоритм валидации SD, использующий динамическое программирование с применением деревьев Фенвика и деревьев отрезков, что обеспечивает сложность $O(N \log N)$
3. Алгоритм реализован в ядре проекта Desbordante на языке программирования C++
4. Разработаны Python-привязки
5. Создан пример использования, демонстрирующий валидацию SD на данных сетевого мониторинга с автоматическим исправлением набора данных

Реализация доступна в pull request #699 на GitHub¹.

¹<https://github.com/Desbordante/desbordante-core/pull/699>

Список литературы

- [1] Chernishev George, Polyntsov Michael, Chizhov Anton, Stupakov Kirill, Shchuckin Ilya, Smirnov Alexander, Strutovsky Maxim, Shlyonskikh Alexey, Firsov Mikhail, Manannikov Stepan, Bobrov Nikita, Goncharov Daniil, Barutkin Ilia, Yakshigulov Vadim, Shalnev Vladislav, Muraviev Kirill, Rakhmukova Anna, Shcheka Dmitriy, Chernikov Anton, Kuzin Yakov, Sinelnikov Michael, Abrosimov Grigorii, Popov Dmitriy, Demchenko Artem, Belokonny Sergey, Soloveva Liana-Iuliia, Kurbatov Yaroslav, Vyrodov Mikhail, Saliou Arthur, Gaisin Eduard, and Smirnov Kirill. [Desbordante: from benchmarking suite to high-performance science-intensive data profiler](#) // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — New York, NY, USA : Association for Computing Machinery. — 2025. — CODS-COMAD '24. — P. 234–243. — Access mode: <https://doi.org/10.1145/3703323.3703725>.
- [2] Jakob Wenzel, Rhinelander Jason, and Moldovan Dean. `pybind11` — Seamless operability between C++11 and Python. — <https://github.com/pybind/pybind11>. — 2017.
- [3] Kaggle. IoT Telemetry Data. — <https://www.kaggle.com/datasets/ajmal00147/iot-telemetry-data>. — 2020.
- [4] Golab Lukasz, Karloff Howard, Korn Flip, Saha Avishek, and Srivastava Divesh. Sequential Dependencies. // [PVLDB](#). — 2009. — 08. — Vol. 2. — P. 574–585.